

语言处理系统完整流程与 SysY 等价 LLVM IR/RISC-V 程序实验

编译原理 上机作业调研报告

成员 A 姓名（待填写）（成员 A 学号（待填写）） 成员 B 姓名（待填写）（成员 B 学号（待填写））

班级（待填写） 2026 年 10 月

成员分工

根据作业要求，小题 1 由两名成员分别独立完成，小题 2 中的两个任务由两名成员各负责一个。两人共同确定报告整体框架、SysY 示例程序和统一测试数据，并共同维护摘要、引言、结论及参考文献。成员 A 独立完成小题 1 的 GCC 语言处理系统实验，并负责小题 2 的 RISC-V 汇编实现；成员 B 独立完成自己的小题 1，并负责小题 2 的 LLVM IR 实现。

表 1 小组任务分工

内容	成员 A	成员 B
小题 1	独立完成 GCC/Clang 实验并撰写	独立完成 Clang/LLVM 编译流程实验并撰写
共同 SysY 程序	共同设计、共同确定测试数据	共同设计、共同确定测试数据
小题 2 LLVM IR	配合核对等价性	负责编写、链接、运行验证与撰写
小题 2 RISC-V	负责编写、链接、运行验证与撰写	配合核对等价性
共同部分	题目、摘要、关键词、引言、总体方法、结论、参考文献共同完成	

摘要

本报告围绕语言处理系统的完整工作过程以及 SysY 程序到中间代码、目标代码的等价实现开展实验。小题 1 由两名成员分别独立完成，以 GCC 或 LLVM/Clang 为研究对象，通过预处理、编译器内部阶段、汇编和链接等实际输出，分析各阶段的输入、输出及其与源程序的对应关系，并比较不同优化等级下的中间表示。小题 2 采用同一份 SysY 示例程序作为语义基准，程序覆盖数组、赋值、算术与取模运算、条件分支、循环、函数和运行时输入输出等语言特性；成员 B 手写 LLVM IR，成员 A 实现 RISC-V 汇编，并链接 SysY 运行时库。两种实现均使用统一的三组测试输入验证，所得偶数和及正数计数与预期一致。

关键词： GCC；编译流程；SysY；LLVM IR；RISC-V；运行时库

1 引言

高级语言程序从源代码变为可执行程序，并不是由单一阶段一次完成，而是由预处理器、编译器、汇编器和链接器等多个组成部分协同工作。进一步观察编译器内部，还可以划分为词法分析、语法分析、语义分析、中间代码生成、代码优化和目标代码生成等阶段。若只观察最终执行结果，难以直接理解宏、类型、控制流、函数调用、符号和库依赖在这些阶段中如何变化，因此本实验利用编译工具提供的命令行选项保存各阶段产物，并将其与源程序逐项对照。

在此基础上，小题 2 进一步从“观察编译器输出”转向“直接编写中间表示和目标汇编”。两名成员共同设计一份 SysY 示例程序，再分别用 LLVM IR 和 RISC-V 汇编实现相同语义，并链接 SysY 运行时库形成目标程序。通过统一测试数据比较两种实现的输出，可从基本块、条件跳转、函数调用、数组地址计算、调用约定和运行时库接口等角度理解源语言语义向中间表示和目标代码的映射关系。

2 实验环境与总体方法

成员 A 的小题 1 实验在 WSL2 Ubuntu 22.04 环境中完成，主要使用 GCC 11.4.0，并在词法分析阶段使用 Clang 14 输出 token 序列，目标平台为 x86-64。成员 B 的小题 1 与 LLVM IR 实验在 Ubuntu 22.04 Docker 容器中完成，主要使用 Clang/LLVM 14.0.0、GCC 11.4.0、Graphviz 2.43.0；IR 以 x86-64 Linux 为运行目标，流程实验另观察 AArch64 目标汇编与 ELF 文件。RISC-V 部分使用课程提供的交叉工具链与 QEMU，目标为 RV64，并链接课程提供的 SysY RISC-V 运行时库。

总体方法分为两条主线。第一条主线以一份简单 C/C++ 程序为研究对象，利用编译器选项观察预处理文件、token、GCC tree、CFG、RTL、汇编文件、目标文件和最终可执行文件，并通过参数对照观察不同设置对各阶段输出的影响。第二条主线以同一份 SysY 程序为语义基准，由两名成员分别手工实现 LLVM IR 与 RISC-V 版本，完成编译/汇编、链接和多组输入验证，检查两种实现是否与 SysY 源程序保持等价。

3 小题 1：成员 A 的 GCC 语言处理系统完整流程实验

本小题以一个阶乘 C++ 程序为对象，使用 GCC 工具链中的 g++ 驱动程序，依次观察预处理、编译、汇编和链接阶段的输出。源程序在保持功能简单的同时加入自定义头文件、普通宏、带参宏、条件编译、全局变量、全局常量、函数、条件分支和循环，便于在各阶段观察这些语言成分的变化。实验环境为 WSL2 下的 Ubuntu 22.04，g++ 版本为 11.4.0，目标平台为 x86-64，所有中间文件统一保存在 output 目录中。

3.1 实验环境与源程序

自定义头文件 test.h 定义输入上限 N、带参宏 mul(a,b) 和 DEBUG 条件编译宏; main.cpp 中包含全局变量 times、全局常量 start、阶乘函数 factorial(int n) 以及主函数。

```
C test.h
1  #ifndef TEST_H
2  #define TEST_H
3
4  //宏
5  #define N 20
6
7  //带参宏
8  #define mul(a,b) ((a)*(b))
9
10 //条件编译
11 #ifdef DEBUG
12 #define debug(x) cout << x << endl
13 #else
14 #define debug(x)
15 #endif
16
17 #endif
```

图 1 自定义头文件 test.h

```
main.cpp
1  //头文件
2  #include <iostream>
3  //自定义头文件
4  #include "test.h"
5  using namespace std;
6
7  //全局变量
8  int times = 0;
9  //全局常量
10 const int start = 2;
11
12 //函数
13 long long factorial(int n)
14 {
15     //局部变量
16     long long result = 1;
17
18     //循环
19     for (int i = start; i <= n; i++)
20     {
21         //运算、赋值、宏
22         result = mul(result, i);
23         times++;
24     }
25     return result;
26 }
27
28 int main()
29 {
30     int n;
31
32     //输入
33     cin >> n;
34     //条件、逻辑运算
35     if (n < 0 || n > N)
36     {
37         return 0;
38     }
39     //条件编译
40     debug(n);
41     //函数调用、输出
42     cout << factorial(n) << endl;
43     cout << times << endl;
44
45     return 0;
46 }
```

图 2 主程序 main.cpp

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ pwd
ls
g++ --version
/mnt/d/compilerLab
image main.cpp output test.h
g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 3 实验目录与 GCC 版本

先直接编译并运行源程序，确认后续实验建立在正确程序上：

```
1 g++ main.cpp -o main
2 ./main
```

输入 5 后得到 120 和 1，其中 120 为 5!，1 为 factorial 函数调用次数。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ main.cpp -o main
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ./main
5
120
1
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 4 基准程序运行结果

3.2 预处理器

使用 -E 选项只执行预处理，并把结果保存为 main.ii：

```
1 g++ -E main.cpp -o output/main.ii
```

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -E main.cpp -o output/main.ii
i
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls output
main.ii
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 5 生成预处理文件 main.ii

预处理器首先处理 #include。检查 main.ii 可以看到 iostream 的内容已经进入预处理结果，同时保留了 test.h 的文件切换标记，说明标准头文件和自定义头文件都已经被处理。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "iostream" output/main.i
i | head
8:# 1 "/usr/include/c++/11/iostream" 1 3
9:# 36 "/usr/include/c++/11/iostream" 3
11:# 37 "/usr/include/c++/11/iostream" 3
73:# 39 "/usr/include/c++/11/iostream" 2 3
1217:     class basic_iostream;
1272: typedef basic_iostream<char> iostream;
1312: typedef basic_iostream<wchar_t> wiostream;
25106: const error_category& iostream_category() noexcept;
25110: { return error_code(static_cast<int>(__e), iostream_category()); }
25114: { return error_condition(static_cast<int>(__e), iostream_category()
); }
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "test.h" output/main.ii
| head
32255:# 1 "test.h" 1
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 6 头文件展开后的内容

随后检查宏展开结果。普通宏 `N` 被直接替换为 `20`, `mul(result,i)` 被展开为 `((result)*(i))`; 源文件中的注释已经消失。未定义 `DEBUG` 时, `debug(n)` 展开为空, 因此对应位置只剩一个分号。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ tail -n 50 output/main.ii

static ios_base::Init __ioinit;

}
# 3 "main.cpp" 2

# 1 "test.h" 1
# 5 "main.cpp" 2

# 5 "main.cpp"
using namespace std;

int times = 0;

const int start = 2;

long long factorial(int n)
{
    long long result = 1;

    for (int i = start; i <= n; i++)
        result = ((result)*(i));
    times++;
    return result;
}

int main()
{
    int n;

    cin >> n;

    if (n < 0 || n > 20)
        return 0;

    ;

    cout << factorial(n) << endl;
    cout << times << endl;

    return 0;
}
```

图 7 未定义 DEBUG 时的预处理结果

在命令行加入 `-DDEBUG` 后重新预处理，条件编译选择 `DEBUG` 分支，原来的 `debug(n)` 展开为 `cout << n << endl`:

```
1 g++ -E -DDEBUG main.cpp -o output/main_debug.ii
```

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -E -DDEBUG main.cpp -o output/main_debug.ii
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ tail -n 50 output/main_debug.ii

    static ios_base::Init __ioint;

}
# 3 "main.cpp" 2

# 1 "test.h" 1
# 5 "main.cpp" 2

# 5 "main.cpp"
using namespace std;

int times = 0;

const int start = 2;

long long factorial(int n)
{
    long long result = 1;

    for (int i = start; i <= n; i++)
        result = ((result)*(i));
    times++;
    return result;
}

int main()
{
    int n;

    cin >> n;

    if (n < 0 || n > 20)
        return 0;

    cout << n << endl;
```

图 8 定义 DEBUG 后的预处理结果

预处理后的文件规模明显增大:main.cpp 为 41 行、test.h 为 16 行,而 main.ii 达到 32296 行、约 761 KB。主要原因是 iostream 及其依赖的标准库头文件被展开到预处理结果中。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ wc -l main.cpp test.h output/main.ii
  41 main.cpp
  16 test.h
32296 output/main.ii
32353 total
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh main.cpp test.h output/main.ii
-rwxrwxrwx 1 lenovo lenovo  695 Sep 22 09:11 main.cpp
-rwxrwxrwx 1 lenovo lenovo 761K Sep 22 10:03 output/main.ii
-rwxrwxrwx 1 lenovo lenovo  207 Sep 22 09:30 test.h
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 9 源文件与预处理文件规模比较

3.3 编译器内部阶段

编译器内部可进一步分为词法分析、语法分析、语义分析、中间代码生成、代码优化和代码生成。本实验主要使用 GCC 的 `tree`、`CFG`、`RTL` 与最终汇编结果观察编译器内部处理，并在词法分析阶段使用 Clang 输出 token 序列作为补充。

3.3.1 词法分析

使用 Clang 对原有 `main.cpp` 输出 token 序列。为避免标准头文件产生大量无关输出，只保留来源于 `main.cpp` 的 token：

```
1 clang++ -Xclang -dump-tokens -fsyntax-only main.cpp 2>&1 \  
2 | grep "main.cpp:" | head -n 60
```

输出中可以看到 `using`、`int`、`const`、`for` 等关键字，`times`、`factorial` 等标识符，数值常量和运算符也被识别为独立 token。宏展开得到的 token 还保留了在 `test.h` 中的来源信息，说明此阶段已将源程序字符序列转换为词法单元序列。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ cd /mnt/d/compilerLab
clang++ -Xclang -dump-tokens -fsyntax-only main.cpp 2>&1 | grep "main.cpp:" | head -n 60
using 'using'          Loc=<main.cpp:5:1>
namespace 'namespace'  [LeadingSpace] Loc=<main.cpp:5:7>
identifier 'std'        [LeadingSpace] Loc=<main.cpp:5:17>
semi ';'               Loc=<main.cpp:5:20>
int 'int'               [StartOfLine]  Loc=<main.cpp:8:1>
identifier 'times'      [LeadingSpace] Loc=<main.cpp:8:5>
equal '='               [LeadingSpace] Loc=<main.cpp:8:11>
numeric_constant '0'    [LeadingSpace] Loc=<main.cpp:8:13>
semi ';'               Loc=<main.cpp:8:14>
const 'const'          [StartOfLine]  Loc=<main.cpp:10:1>
int 'int'               [LeadingSpace] Loc=<main.cpp:10:7>
identifier 'start'      [LeadingSpace] Loc=<main.cpp:10:11>
equal '='               [LeadingSpace] Loc=<main.cpp:10:17>
numeric_constant '2'    [LeadingSpace] Loc=<main.cpp:10:19>
semi ';'               Loc=<main.cpp:10:20>
long 'long'             [StartOfLine]  Loc=<main.cpp:13:1>
long 'long'             [LeadingSpace] Loc=<main.cpp:13:6>
identifier 'factorial'  [LeadingSpace] Loc=<main.cpp:13:11>
l_paren '('             Loc=<main.cpp:13:20>
int 'int'               Loc=<main.cpp:13:21>
identifier 'n'          [LeadingSpace] Loc=<main.cpp:13:25>
r_paren ')'             Loc=<main.cpp:13:26>
l_brace '{'             [StartOfLine]  Loc=<main.cpp:14:1>
long 'long'             [StartOfLine]  [LeadingSpace] Loc=<main.cpp:16:5>
long 'long'             [LeadingSpace] Loc=<main.cpp:16:10>
identifier 'result'     [LeadingSpace] Loc=<main.cpp:16:15>
equal '='               [LeadingSpace] Loc=<main.cpp:16:22>
numeric_constant '1'    [LeadingSpace] Loc=<main.cpp:16:24>
semi ';'               Loc=<main.cpp:16:25>
for 'for'               [StartOfLine]  [LeadingSpace] Loc=<main.cpp:19:5>
l_paren '('             [LeadingSpace] Loc=<main.cpp:19:9>
int 'int'               Loc=<main.cpp:19:10>
identifier 'i'          [LeadingSpace] Loc=<main.cpp:19:14>
equal '='               [LeadingSpace] Loc=<main.cpp:19:16>
identifier 'start'      [LeadingSpace] Loc=<main.cpp:19:18>
semi ';'               Loc=<main.cpp:19:23>
identifier 'i'          [LeadingSpace] Loc=<main.cpp:19:25>
lessequal '<='          [LeadingSpace] Loc=<main.cpp:19:27>
identifier 'n'          [LeadingSpace] Loc=<main.cpp:19:30>
semi ';'               Loc=<main.cpp:19:31>
identifier 'i'          [LeadingSpace] Loc=<main.cpp:19:33>
plusplus '++'           Loc=<main.cpp:19:34>
r_paren ')'             Loc=<main.cpp:19:36>
identifier 'result'     [StartOfLine]  [LeadingSpace] Loc=<main.cpp:21:9>
equal '='               [LeadingSpace] Loc=<main.cpp:21:16>
l_paren '('             [LeadingSpace] Loc=<main.cpp:21:18 <Spelling=./test.h:8:18>>
l_paren '('             Loc=<main.cpp:21:18 <Spelling=./test.h:8:19>>

```

图 10 Clang 输出的 main.cpp 词法 token 序列

3.3.2 语法分析

使用 `-fdump-tree-original-raw` 输出 GCC 的 tree 表示:

```
1 g++ -fdump-tree-original-raw -c main.cpp -o output/main_ast.o
```

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -fdump-tree-original-raw -c main.cpp -o output/main_ast.o
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls
image  main  main.cpp  output  test.h
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls output
main.ii  main_ast.cpp.005t.original  main_ast.o  main_debug.ii
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 11 生成 GCC tree 输出文件

在 factorial 函数对应的 tree 中可以看到 `function_decl`、`parm_decl`、`var_decl`、`identifier_node`、`statement_list`、`return_expr` 等节点。源程序中的函数、参数、局部变量和语句已被组织为树结构，而不再按原始 C++ 文本保存。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ sed -n '61252,61340p' output/main_ast.cpp.005t.original
;; Function long long int factorial(int) (null)
;; enabled by -tree-original

@1      bind_expr      type: @2      vars: @3      body: @4
@2      void_type      name: @5      algn: 8
@3      var_decl       name: @6      type: @7      scpe: @8
                        srcp: main.cpp:16      init: @9
                        size: @10      algn: 64      used: 1
@4      statement_list 0 : @11      1 : @12      2 : @13
                        3 : @14
@5      type_decl      name: @15      type: @2      srcp: <built-in>:0
                        note: artificial
@6      identifier_node strg: result  lngt: 6
@7      integer_type   name: @16      size: @10      algn: 64
                        prec: 64      sign: signed  min : @17
                        max : @18
@8      function_decl  name: @19      type: @20      scpe: @21
                        srcp: main.cpp:13      args: @22
                        link: extern
@9      integer_cst    type: @7      int: 1
@10     integer_cst    type: @23      int: 64
@11     cleanup_point_expr type: @2      op 0: @24
@12     bind_expr      type: @2      vars: @25      body: @26
@13     cleanup_point_expr type: @2      op 0: @27
@14     return_expr    type: @2      expr: @28
@15     identifier_node strg: void    lngt: 4
@16     type_decl      name: @29      type: @7      srcp: <built-in>:0
                        note: artificial
@17     integer_cst    type: @7      int: -9223372036854775808
@18     integer_cst    type: @7      int: 9223372036854775807
@19     identifier_node strg: factorial lngt: 9
@20     function_type   size: @30      algn: 8      retn: @7
                        prms: @31
@21     translation_unit_decl name: @32
@22     parm_decl      name: @33      type: @34      scpe: @8
                        srcp: main.cpp:13      argt: @34
                        size: @35      algn: 32      used: 1
@23     integer_type   name: @36      size: @37      algn: 128
                        prec: 128      sign: unsigned min : @38
                        max : @39
@24     decl_expr      type: @2
@25     var_decl       name: @40      type: @34      scpe: @8
                        srcp: main.cpp:19      init: @41
                        size: @35      algn: 32      used: 1
@26     statement_list 0 : @42      1 : @43
@27     expr_stmt      type: @2      line: 22
@28     init_expr      type: @7      op 0: @45      op 1: @3
@29     identifier_node strg: long long int lngt: 13
@30     integer_cst    type: @23      int: 8
@31     tree_list      valu: @34      chan: @46

```

图 12 factorial 函数的 GCC tree 片段

3.3.3 语义分析

为观察类型检查，复制源程序后只修改一次函数调用，把 `factorial(n)` 改为 `factorial("5")`。语法形式仍是函数调用，但实参类型从 `int` 变为 `const char*`。

```
//函数调用、输出
cout << factorial("5") << endl;
cout << times << endl;
```

图 13 故意修改的函数调用

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -c main_error.cpp -o output/main_error.o
main_error.cpp: In function 'int main()':
main_error.cpp:38:23: error: invalid conversion from 'const char*' to 'int' [-fpermissive]
   38 |     cout << factorial("5") << endl;
      |                      ^~~
      |                      |
      |                  const char*
main_error.cpp:13:25: note: initializing argument 1 of 'long long int factorial(int)'
   13 | long long factorial(int n)
      |                   ~~~~~^
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 14 GCC 类型检查报错

GCC 报告 invalid conversion from 'const char*' to 'int', 并指出 factorial(int) 的参数类型为 int。这说明编译器不仅检查语法结构, 还会利用声明和类型信息进行语义检查。

3.3.4 中间代码生成

使用 -fdump-tree-all-graph 输出 GCC 多个 tree 阶段的 .dot 文件:

```
1 g++ -O0 -fdump-tree-all-graph -c main.cpp -o output/main_tree.o
```

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 -fdump-tree-all-graph -c main.cpp -o output/main_tree.o
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ find . -maxdepth 2 -type f -name "*.dot" | sort | head -n 20
./output/main_tree.cpp.015t.cfg.dot
./output/main_tree.cpp.017t.ompexp.dot
./output/main_tree.cpp.018t.warn-printf.dot
./output/main_tree.cpp.022t.fixup_cfg1.dot
./output/main_tree.cpp.023t.ssa.dot
./output/main_tree.cpp.027t.fixup_cfg2.dot
./output/main_tree.cpp.028t.local-fnsummary1.dot
./output/main_tree.cpp.029t.inline.dot
./output/main_tree.cpp.051t.release_ssa.dot
./output/main_tree.cpp.052t.local-fnsummary2.dot
./output/main_tree.cpp.092t.fixup_cfg3.dot
./output/main_tree.cpp.093t.ehdisp.dot
./output/main_tree.cpp.097t.adjust_alignment.dot
./output/main_tree.cpp.233t.veclower.dot
./output/main_tree.cpp.234t.cplxlower0.dot
./output/main_tree.cpp.236t.switchlower_00.dot
./output/main_tree.cpp.240t.ehcleanup2.dot
./output/main_tree.cpp.241t.resx.dot
./output/main_tree.cpp.243t.isel.dot
./output/main_tree.cpp.244t.optimized.dot
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 15 Tree/CFG 多阶段输出文件

将较早的 `cfg.dot` 转成图片后，可以直接看到基本块及其控制流关系。`factorial` 中的 `for` 循环被拆成初始化、条件判断、循环体、回跳和退出等基本块；`main` 中的输入、范围判断、函数调用和输出也形成了清晰的分支结构。

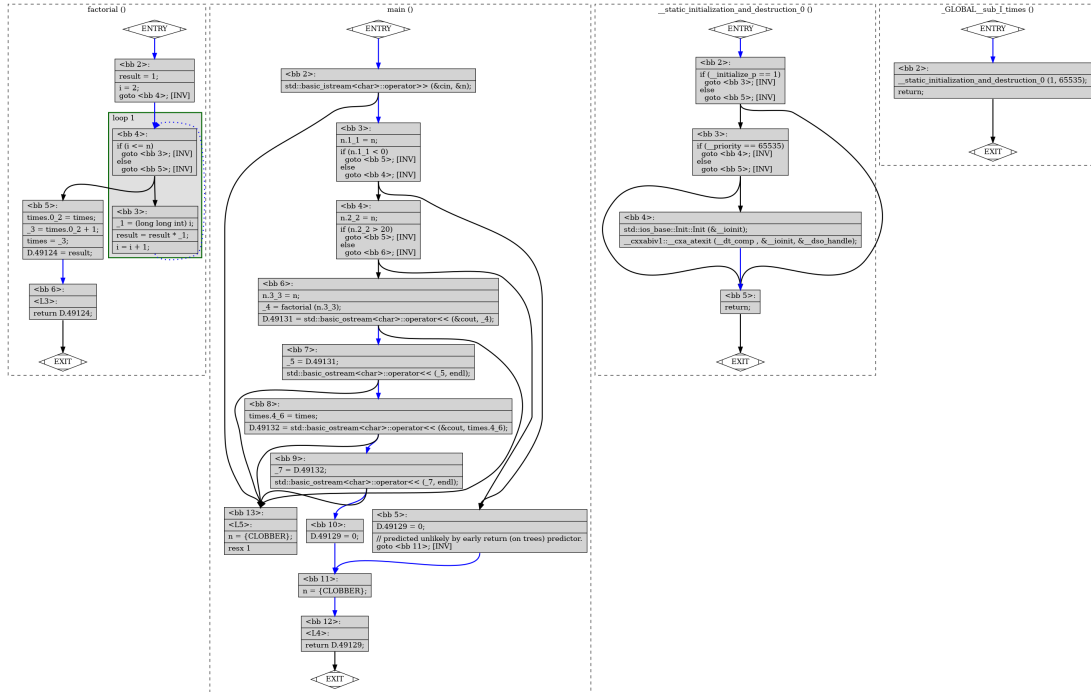


图 16 GCC 生成的 CFG

继续使用 `-fdump-rtl-all-graph` 输出 RTL 多阶段结果：

```
1 g++ -O0 -fdump-rtl-all-graph -c main.cpp -o output/main_rtl.o
```

RTL 比 `tree` 更接近目标机器，其中已经出现寄存器、调用、比较和跳转等低层表示，可读性明显下降。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 -fdump-rtl-all-graph -c main.cpp -o output/main_rtl.o
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ find output -maxdepth 1 -type f -name "main_rtl*.dot" | sort | head -n 30
output/main_rtl.cpp.245r.expand.dot
output/main_rtl.cpp.246r.vregs.dot
output/main_rtl.cpp.247r.into_cfglayout.dot
output/main_rtl.cpp.248r.jump.dot
output/main_rtl.cpp.260r.reginfo.dot
output/main_rtl.cpp.284r.outof_cfglayout.dot
output/main_rtl.cpp.285r.split1.dot
output/main_rtl.cpp.287r.dfini.dot
output/main_rtl.cpp.288r.mode_sw.dot
output/main_rtl.cpp.289r.asmcons.dot
output/main_rtl.cpp.294r.ira.dot
output/main_rtl.cpp.295r.reload.dot
output/main_rtl.cpp.302r.pro_and_epilogue.dot
output/main_rtl.cpp.305r.jump2.dot
output/main_rtl.cpp.316r.split4.dot
output/main_rtl.cpp.317r.stack.dot
output/main_rtl.cpp.318r.zero_call_used_regs.dot
output/main_rtl.cpp.319r.alignments.dot
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 17 RTL 多阶段输出文件

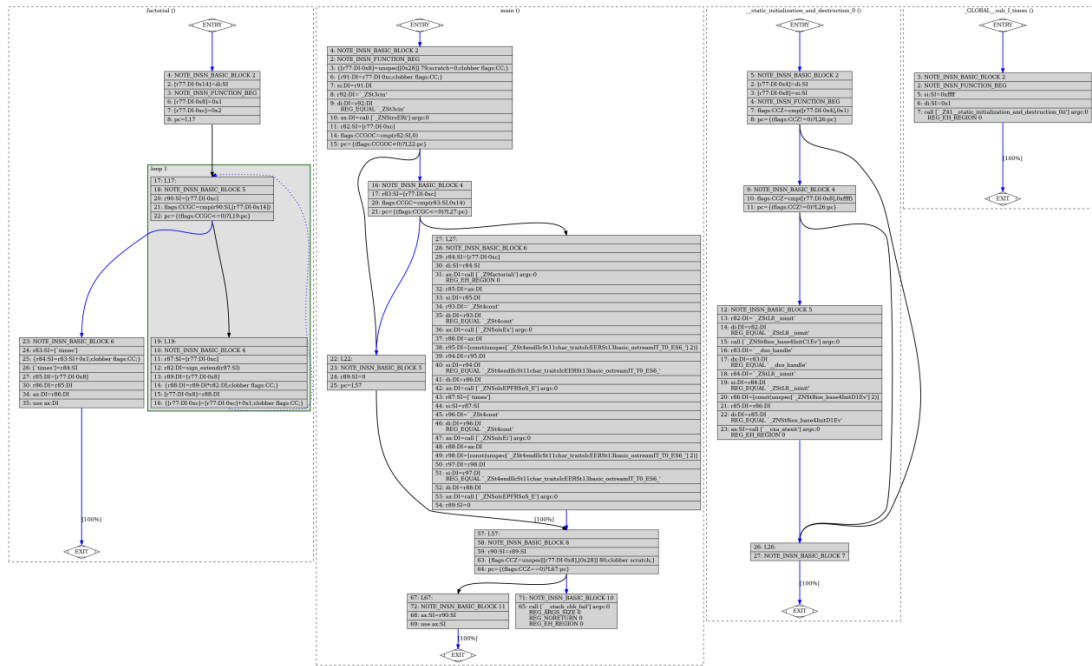


图 18 RTL expand 阶段控制流图

3.3.5 代码优化

使用相同的预处理结果分别生成 -O0 和 -O2 汇编文件，并比较输出：

```
1 g++ -O0 -S -masm=att output/main.ii -o output/main_00.s
2 g++ -O2 -S -masm=att output/main.ii -o output/main_02.s
```

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O2 -S -masm=att output/main.ii -o output/main_02.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_00.s output/main_02.s
-rwxrwxrwx 1 lenovo lenovo 3.6K Sep 24 15:45 output/main_00.s
-rwxrwxrwx 1 lenovo lenovo 4.2K Sep 24 15:55 output/main_02.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ wc -l output/main_00.s output/main_02.s
193 output/main_00.s
222 output/main_02.s
415 total
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "factorial" output/main_02.s
73:      .globl _Z9factoriali
74:      .type _Z9factoriali, @function
75:_Z9factoriali:
103:     .size _Z9factoriali, .-_Z9factoriali
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 19 -O0 与 -O2 汇编文件比较

整体文件大小由 3.6 KB 增至 4.2 KB，行数由 193 行增至 222 行，因此不能把“优

化”简单理解为汇编文本一定更短。观察 factorial 函数本身，-O2 版本主要使用寄存器保存循环变量和中间结果，循环体缩短为 imulq、addq、cmpq、jne 等少量指令，同时 times++ 被压缩为对内存的直接加法。与 -O0 相比，栈内存读写明显减少。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ sed -n '73,103p' output/main_O2
.s
    .globl _Z9factoriali
    .type _Z9factoriali, @function
_Z9factoriali:
.LFB1745:
    .cfi_startproc
    endbr64
    cmpl    $1, %edi
    jle     .L13
    leal    1(%rdi), %edx
    movl    $2, %eax
    movl    $1, %r8d
    .p2align 4,,10
    .p2align 3
.L12:
    imulq   %rax, %r8
    addq    $1, %rax
    cmpq    %rdx, %rax
    jne     .L12
    addl    $1, times(%rip)
    movq    %r8, %rax
    ret
    .p2align 4,,10
    .p2align 3
.L13:
    movl    $1, %r8d
    addl    $1, times(%rip)
    movq    %r8, %rax
    ret
    .cfi_endproc
.LFE1745:
    .size _Z9factoriali, .-_Z9factoriali
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 20 -O2 下 factorial 函数的汇编结果

3.3.6 代码生成

使用 -S 让 GCC 在生成汇编代码后停止，得到 x86-64 AT&T 语法的 main_00.s。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 -S -masm=att output/main.ii -o output/main_00.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main.ii output/main_00.s
-rwxrwxrwx 1 lenovo lenovo 761K Sep 22 10:03 output/main.ii
-rwxrwxrwx 1 lenovo lenovo 3.6K Sep 24 15:45 output/main_00.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 21 由 main.ii 生成汇编文件 main_00.s

在未优化版本中, factorial 函数仍能与源程序逐项对应: movq \$1, -8(%rbp) 对应 result=1, movl \$2, -12(%rbp) 对应循环初值, imulq 对应乘法, cmpl/jle 对应循环条件, 最后把 result 放入 %rax 后 ret 返回。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ sed -n '19,54p' output/main_00.s
s
        .globl  _Z9factoriali
        .type   _Z9factoriali, @function
_Z9factoriali:
.LFB1731:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    %edi, -20(%rbp)
        movq    $1, -8(%rbp)
        movl    $2, -12(%rbp)
        jmp     .L2
.L3:
        movl    -12(%rbp), %eax
        cltq
        movq    -8(%rbp), %rdx
        imulq   %rdx, %rax
        movq    %rax, -8(%rbp)
        addl    $1, -12(%rbp)
.L2:
        movl    -12(%rbp), %eax
        cmpl    -20(%rbp), %eax
        jle     .L3
        movl    times(%rip), %eax
        addl    $1, %eax
        movl    %eax, times(%rip)
        movq    -8(%rbp), %rax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE1731:
        .size   _Z9factoriali, .-_Z9factoriali
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 22 -O0 下 factorial 函数的汇编结果

main 函数中可以看到 cin、cout 对应的库函数调用,以及 call _Z9factoriali。
_Z9factoriali 是 C++ 对 factorial(int) 进行名字修饰后的符号名。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ sed -n '57,105p' output/main_00.s
main:
.LFB1732:
.cfi_startproc
endbr64
pushq   %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq    %rsp, %rbp
.cfi_def_cfa_register 6
subq    $16, %rsp
movq    %fs:40, %rax
movq    %rax, -8(%rbp)
xorl    %eax, %eax
leaq    -12(%rbp), %rax
movq    %rax, %rsi
leaq    _ZSt3cin(%rip), %rax
movq    %rax, %rdi
call    _ZNSirsERi@PLT
movl    -12(%rbp), %eax
testl   %eax, %eax
js       .L6
movl    -12(%rbp), %eax
cmpl    $20, %eax
jle      .L7
.L6:
movl    $0, %eax
jmp      .L9
.L7:
movl    -12(%rbp), %eax
movl    %eax, %edi
call    _Z9factoriali
movq    %rax, %rsi
leaq    _ZSt4cout(%rip), %rax
movq    %rax, %rdi
call    _ZNSolsEx@PLT
movq    _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_0_ES6_@GOTPCREL(%rip), %rdx
movq    %rdx, %rsi
movq    %rax, %rdi
call    _ZNSolsEPFRSoS_E@PLT
movl    times(%rip), %eax
movl    %eax, %esi
leaq    _ZSt4cout(%rip), %rax
movq    %rax, %rdi
call    _ZNSolsEi@PLT
movq    _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_0_ES6_@GOTPCREL(%rip), %rdx
movq    %rdx, %rsi
movq    %rax, %rdi
call    _ZNSolsEPFRSoS_E@PLT
movl    $0, %eax
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 23 main 函数中的输入、判断、函数调用和输出

3.4 汇编器

使用 `-c` 将汇编文件转换为可重定位目标文件：

```
1 g++ -c output/main_00.s -o output/main_00.o
```

`file` 显示 `main_00.o` 为 ELF 64-bit LSB relocatable, x86-64, 说明汇编器已把文本汇编指令编码为机器指令，但此时文件仍不能独立执行。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -c output/main_00.s -o output/main_00.o
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_00.s output/main_00.o
-rwxrwxrwx 1 lenovo lenovo 3.4K Sep 24 16:07 output/main_00.o
-rwxrwxrwx 1 lenovo lenovo 3.6K Sep 24 15:45 output/main_00.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ file output/main_00.o
output/main_00.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 24 生成可重定位目标文件 main_00.o

使用 `objdump -d` 对目标文件反汇编。左侧十六进制字节是目标文件中的机器码，右侧是反汇编得到的 x86 指令，例如 `eb 15` 对应 `jmp`，`7e e3` 对应 `jle`。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ objdump -d output/main_00.o > output/main_00_disasm.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_00_disasm.s
-rwxrwxrwx 1 lenovo lenovo 5.6K Sep 24 16:09 output/main_00_disasm.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "factorial" output/main_00_disasm.s
7:0000000000000000 <_Z9factoriali>:
15: 1a:      eb 15          jmp     31 <_Z9factoriali+0x31>
24: 37:      7e e3          jle     1c <_Z9factoriali+0x1c>
25: 39:      8b 05 00 00 00 00  mov     0x0(%rip),%eax      # 3f
<_Z9factoriali+0x3f>
27: 42:      89 05 00 00 00 00  mov     %eax,0x0(%rip)      # 48
<_Z9factoriali+0x48>
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 25 目标文件 main_00.o 的反汇编结果

使用 `nm` 查看符号表。T 表示定义在代码段中的全局符号，`main` 和 `_Z9factoriali` 均已在当前目标文件中定义；B 表示 BSS 段中的零初始化全局变量 `times`；U 表示当前目标文件尚未定义、需要在链接阶段由外部库解决的符号，例如 `cin`、`cout` 相关符号。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ nm output/main_00.o > output/main_00_nm.txt
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -E "factorial|times|main" output/main_00_nm.txt
0000000000000015d t _GLOBAL__sub_I_times
0000000000000000 T _Z9factoriali
000000000000004e T main
0000000000000000 B times
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ head -n 30 output/main_00_nm.txt
t
          U _GLOBAL_OFFSET_TABLE_
0000000000000015d t _GLOBAL__sub_I_times
00000000000000107 t _Z41__static_initialization_and_destruction_0ii
0000000000000000 T _Z9factoriali
0000000000000000 r _ZL5start
          U _ZNSirsERi
          U _ZNSolsEPFRSoS_E
          U _ZNSolsEi
          U _ZNSolsEx
          U _ZNSt8ios_base4InitC1Ev
          U _ZNSt8ios_base4InitD1Ev
          U _ZSt3cin
          U _ZSt4cout
          U _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_
0000000000000004 b _ZStL8__ioinit
          U __cxa_atexit
          U __dso_handle
          U __stack_chk_fail
000000000000004e T main
0000000000000000 B times
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 26 目标文件 main_00.o 的符号表

3.5 链接器

使用 g++ 把 main_00.o 与 C++ 运行库等依赖链接为最终可执行文件。由于源程序为 C++，使用 g++ 可以自动加入所需的 C++ 标准库：

```
g++ output/main_00.o -o output/main_00
```

file 显示生成文件为 ELF 64-bit LSB PIE executable，并采用动态链接。运行后输入 5，输出仍为 120 和 1。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ output/main_00.o -o output/main_00
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ file output/main_00

output/main_00: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=28f9861658df36d41be9b8b18706ef48911ed5b4, for GNU/Linux 3.2.0, not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ./output/main_00
5
120
1
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 27 生成并运行动态链接可执行文件

对最终可执行文件再次反汇编，可以看到 `factorial` 和 `main` 已获得链接后的确定地址；`main` 中对 `factorial` 的调用也已经指向链接后的地址。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ objdump -d output/main_00 > output/main_00_exe_disasm.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "factorial" output/main_00_exe_disasm.s | head
169:0000000000001209 <_Z9factoriali>:
177:    1223:    eb 15                jmp     123a <_Z9factoriali+0x31>
186:    1240:    7e e3                jle     1225 <_Z9factoriali+0x1c>
218:    12a3:    e8 61 ff ff ff      call    1209 <_Z9factoriali>
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "<main>" output/main_00_exe_disasm.s | head
109:    1138:    48 8d 3d 18 01 00 00  lea     0x118(%rip),%rdi        # 1
257 <main>
194:0000000000001257 <main>:
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 28 链接后可执行文件的反汇编结果

最终可执行文件的符号表中，`factorial`、`main` 和 `times` 均带有确定地址。由于该文件采用动态链接，部分标准库符号仍以带版本信息的 U 项出现，这些符号由动态链接器和共享库在装载、运行过程中解析。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ nm output/main_00 > output/main_00_exe_nm.txt
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -E "factorial|times|main" output/main_00_exe_nm.txt
0000000000001366 t _GLOBAL__sub_I_times
0000000000001209 T _Z9factoriali
                U __libc_start_main@GLIBC_2.34
0000000000001257 T main
000000000000427c B times
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep " U " output/main_00_exe_nm.txt | head -n 20
                U _ZNSirsERi@GLIBCXX_3.4
                U _ZNSolsEPFRSoS_E@GLIBCXX_3.4
                U _ZNSolsEi@GLIBCXX_3.4
                U _ZNSolsEx@GLIBCXX_3.4
                U _ZNSt8ios_base4InitC1Ev@GLIBCXX_3.4
                U _ZNSt8ios_base4InitD1Ev@GLIBCXX_3.4
                U _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_@GLIBCXX_3.4
                U __cxa_atexit@GLIBC_2.2.5
                U __libc_start_main@GLIBC_2.34
                U __stack_chk_fail@GLIBC_2.4
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 29 链接后可执行文件的符号表

进一步使用 `-static` 生成静态链接版本。动态链接版本约 17 KB，静态链接版本约 2.3 MB，差异来自静态版本把更多库代码直接并入可执行文件。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -static output/main_00.o -o output/main_static

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ file output/main_static
output/main_static: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=f31cdc942fc7e8f7ded64835673bfc424bbc2f3a, for GNU/Linux 3.2.0, not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_00 output/main_static
-rwxrwxrwx 1 lenovo lenovo 17K Sep 24 17:01 output/main_00
-rwxrwxrwx 1 lenovo lenovo 2.3M Sep 24 19:39 output/main_static
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 30 动态链接与静态链接文件大小比较

静态链接版本运行同样得到 120 和 1，说明链接方式改变了库代码的组织 and 装载方式，没有改变程序的计算结果。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ./output/main_static
5
120
1
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 31 静态链接程序运行结果

3.6 编译参数扩展探索

3.6.1 调试信息 -g

在相同 -O0 条件下分别生成不带和带 -g 的可执行文件。main_nog 为 17 KB, main_g 为 34 KB; readelf 检查显示不带 -g 的文件没有 .debug 段, 而带 -g 的文件包含 .debug_info、.debug_line、.debug_str 等调试信息。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 main.cpp -o output/main_nog
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 -g main.cpp -o output/main_g
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_nog output/main_g
-rwxrwxrwx 1 lenovo lenovo 34K Sep 24 20:40 output/main_g
-rwxrwxrwx 1 lenovo lenovo 17K Sep 24 20:40 output/main_nog
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ echo "=== no -g ==="
readelf -S output/main_nog | grep "\.debug" || echo "no debug sections"

echo "=== with -g ==="
readelf -S output/main_g | grep "\.debug"
=== no -g ===
no debug sections
=== with -g ===
[28] .debug_aranges    PROGBITS          0000000000000000  0000303d
[29] .debug_info        PROGBITS          0000000000000000  0000306d
[30] .debug_abbrev       PROGBITS          0000000000000000  00005559
[31] .debug_line         PROGBITS          0000000000000000  00005b77
[32] .debug_str          PROGBITS          0000000000000000  00005d2b
[33] .debug_line_str     PROGBITS          0000000000000000  00006f79
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 32 -g 对文件大小和调试段的影响

3.6.2 -fno-omit-frame-pointer

在相同 -O2 优化级别下加入 -fno-omit-frame-pointer。普通 -O2 版本主要以 %rsp 为基准访问局部数据; 加入该选项后出现 movq %rsp,%rbp, 并改用 %rbp 为基准访问局部变量, 说明该参数改变了栈帧和寄存器分配方式。

```

=== O2 ===
main:
.LFB1813:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    leaq    _ZSt3cin(%rip), %rdi
    subq    $16, %rsp
    .cfi_def_cfa_offset 32
    movq    %fs:40, %rax
    movq    %rax, 8(%rsp)
    xorl    %eax, %eax
    leaq    4(%rsp), %rsi
    call    _ZNSirsERI@PLT
    movl    4(%rsp), %eax
    cmpl    $20, %eax
    ja      .L16
    cmpl    $1, %eax
    jle     .L20
    leal    -2(%rax), %edx
=== O2 + -fno-omit-frame-pointer ===
main:
.LFB1813:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    leaq    _ZSt3cin(%rip), %rdi
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    pushq   %r12
    leaq    -28(%rbp), %rsi
    subq    $24, %rsp
    .cfi_offset 12, -24
    movq    %fs:40, %rax
    movq    %rax, -24(%rbp)
    xorl    %eax, %eax
    call    _ZNSirsERI@PLT
    movl    -28(%rbp), %eax
    cmpl    $20, %eax
    ja      .L16
    cmpl    $1, %eax
    jle     .L20
    leal    -2(%rax), %edx
    movl    $1, %esi
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 33 -fno-omit-frame-pointer 对 main 栈帧的影响

3.6.3 自动并行化与优化报告

使用 `-O3 -ftree-parallelize-loops=4` 让 GCC 尝试对循环进行更激进的优化, 同时输出优化报告。报告对阶乘循环给出 `couldn't vectorize loop` 和 `not vectorized` 信息, 生成的汇编中也没有匹配到 `GOMP/parallel` 运行时调用。

从源程序看, `result = mul(result, i)` 使当前迭代的 `result` 依赖前一次迭代的结果, 这种累积关系不具备简单的迭代独立性。此次实验中 GCC 没有把该循环转换为可见的并行运行时调用。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ nl -ba main.cpp | sed -n '17,23p'
echo "==== GCC optimization report ====="
grep "main.cpp:19:27: missed: couldn't vectorize loop" output/parallel_report.txt
echo "==== parallel runtime ====="
grep -E "GOMP|parallel" output/main_parallel.s || echo "no parallel runtime calls"
17
18     //循环
19     for (int i = start; i <= n; i++)
20         //运算、赋值、宏
21         result = mul(result, i);
22     times++;
23     return result;
==== GCC optimization report =====
main.cpp:19:27: missed: couldn't vectorize loop
main.cpp:21:18: missed: not vectorized: relevant stmt not supported: _1 = (long long int) i_15;
main.cpp:19:27: missed: couldn't vectorize loop
main.cpp:21:18: missed: not vectorized: relevant stmt not supported: _13 = (long long int) i_23;
==== parallel runtime =====
no parallel runtime calls
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 34 阶乘循环的优化报告与并行化检查

3.7 本小结小结

通过同一个 C++ 阶乘程序，可以把 GCC 工具链的处理过程连续对应起来：main.cpp 经预处理得到 main.i；编译器内部完成词法、语法和语义处理，产生 tree、CFG、RTL 等中间表示，并最终生成 main_00.s；汇编器把 .s 转换为可重定位目标文件 main_00.o；链接器再解析符号和库依赖，生成可执行文件。

对 -DDEBUG、-O2、-g、-fno-omit-frame-pointer、-static 和自动并行化相关选项的对照表明，编译参数会直接影响预处理结果、中间表示、汇编代码、调试信息和最终文件组织方式，而程序的基本功能可以保持不变。

4 小题 1：成员 B 的语言处理系统完整流程实验

本节以成员 B 独立准备的 C17 程序为对象，按“源程序—预处理—前端分析—中间表示—优化—目标代码—目标文件—链接/执行”的顺序观察编译过程。实验在 Ubuntu 22.04 Docker 容器中完成，工具版本为 Clang/LLVM 14.0.0、GCC 11.4.0 和 Graphviz 2.43.0；目标代码生成分别观察 x86-64 LLVM IR 和 AArch64 汇编/目标文件。与成员 A 的阶乘实验不同，本程序通过宏、数组、循环、条件分支和全局变量展示另一组典型结构。

4.1 源程序与预处理

程序读入 1 至 8 个整数。宏 ADJUST(value,bias) 将数组元素乘以比例常数后加上偏置；weighted_sum 根据计算值的奇偶性累加或扣减，并更新全局计数器 processed_items。源文件和头文件保存在仓库的 pipeline/memberB/q1/ 目录中，原始输入为 5 与 1 2 3 4 5。

```
1 #include <stdio.h>
```

```
2
3 #include "config.h"
4
5 const int global_bias = 2;
6 int processed_items = 0;
7
8 int weighted_sum(const int values[], int count)
9 {
10     int total = 0;
11
12     for (int i = 0; i < count; ++i)
13     {
14         int adjusted = ADJUST(values[i], global_bias);
15         if (adjusted % 2 == 0)
16         {
17             total += adjusted;
18         }
19         else
20         {
21             total -= adjusted;
22         }
23         ++processed_items;
24     }
25
26     return total;
27 }
28
29 int main(void)
30 {
31     int count = 0;
32     int values[MAX_ITEMS] = {0};
33
34     if (scanf("%d", &count) != 1 || count < 1 || count > MAX_ITEMS)
35     {
36         return 1;
37     }
38
39     for (int i = 0; i < count; ++i)
40     {
41         if (scanf("%d", &values[i]) != 1)
42         {
```

```

43         return 1;
44     }
45 }
46
47 int result = weighted_sum(values, count);
48 printf("result=%d, processed=%d\n", result, processed_items);
49 return 0;
50 }

```

代码 1 实验使用的 C17 源程序

```

1 #ifndef CONFIG_H
2 #define CONFIG_H
3
4 #define MAX_ITEMS 8
5 #define SCALE_FACTOR 3
6 #define ADJUST(value, bias) ((value) * SCALE_FACTOR + (bias))
7
8 #endif

```

代码 2 宏与数组长度定义

使用预处理选项只执行头文件展开和宏替换，输出写入 main.i:

```

1 mkdir -p build/memberB/q1
2 clang -E -std=c17 -Ipipeline/memberB/q1/include \
3     pipeline/memberB/q1/src/main.c -o build/memberB/q1/main.i

```

预处理结果中，stdio.h 的声明被展开，MAX_ITEMS 在数组声明中替换为 8，ADJUST(values[i], global_bias) 展开为 ((values[i]) * 3 + (global_bias))。因此 .i 文件保留了原有计算逻辑，但已不再依赖这两个宏定义。

4.2 词法、语法与语义分析证据

词法阶段用 Clang 的 token dump 输出 token 种类、词素和源码位置；语法树转储则展示函数、声明、循环、条件分支及表达式的嵌套关系。两个命令都以 -fsyntax-only 避免生成目标文件：

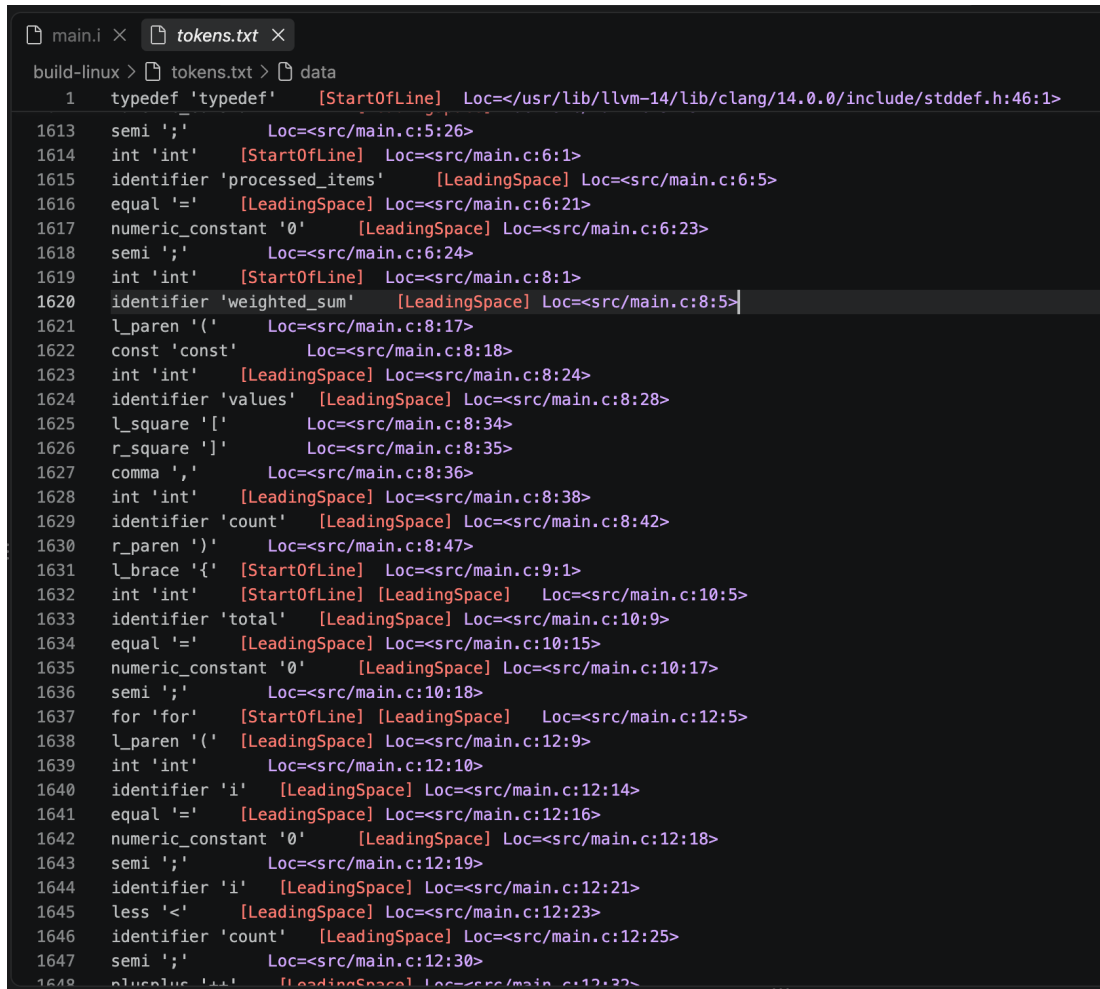
```

1 mkdir -p build/memberB/q1
2 clang -std=c17 -Ipipeline/memberB/q1/include -fsyntax-only \
3     -Xclang -dump-tokens pipeline/memberB/q1/src/main.c \
4     2> build/memberB/q1/tokens.txt
5 clang -std=c17 -Ipipeline/memberB/q1/include -fsyntax-only \
6     -Xclang -ast-dump pipeline/memberB/q1/src/main.c \

```

```
7 | > build/memberB/q1/ast.txt
```

图 35 中可看到 `weighted_sum`、`for`、比较与算术运算符等 token 以及它们在 `src/main.c` 中的行列位置。图 36 展示了 AST 中的 `FunctionDecl`、`ForStmt`、`IfStmt`、`BinaryOperator` 等节点；节点的类型注记也表明 Clang 已完成语法解析和类型语义检查。此处观察的是编译器对合法程序的成功分析，不是单独构造错误程序验证诊断。



```

main.i x tokens.txt x
build-linux > tokens.txt > data
1  typedef 'typedef' [StartOfLine] Loc=<usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:1>
1613 semi ';' Loc=<src/main.c:5:26>
1614 int 'int' [StartOfLine] Loc=<src/main.c:6:1>
1615 identifier 'processed_items' [LeadingSpace] Loc=<src/main.c:6:5>
1616 equal '=' [LeadingSpace] Loc=<src/main.c:6:21>
1617 numeric_constant '0' [LeadingSpace] Loc=<src/main.c:6:23>
1618 semi ';' Loc=<src/main.c:6:24>
1619 int 'int' [StartOfLine] Loc=<src/main.c:8:1>
1620 identifier 'weighted_sum' [LeadingSpace] Loc=<src/main.c:8:5>
1621 l_paren '(' Loc=<src/main.c:8:17>
1622 const 'const' Loc=<src/main.c:8:18>
1623 int 'int' [LeadingSpace] Loc=<src/main.c:8:24>
1624 identifier 'values' [LeadingSpace] Loc=<src/main.c:8:28>
1625 l_square '[' Loc=<src/main.c:8:34>
1626 r_square ']' Loc=<src/main.c:8:35>
1627 comma ',' Loc=<src/main.c:8:36>
1628 int 'int' [LeadingSpace] Loc=<src/main.c:8:38>
1629 identifier 'count' [LeadingSpace] Loc=<src/main.c:8:42>
1630 r_paren ')' Loc=<src/main.c:8:47>
1631 l_brace '{' [StartOfLine] Loc=<src/main.c:9:1>
1632 int 'int' [StartOfLine] [LeadingSpace] Loc=<src/main.c:10:5>
1633 identifier 'total' [LeadingSpace] Loc=<src/main.c:10:9>
1634 equal '=' [LeadingSpace] Loc=<src/main.c:10:15>
1635 numeric_constant '0' [LeadingSpace] Loc=<src/main.c:10:17>
1636 semi ';' Loc=<src/main.c:10:18>
1637 for 'for' [StartOfLine] [LeadingSpace] Loc=<src/main.c:12:5>
1638 l_paren '(' [LeadingSpace] Loc=<src/main.c:12:9>
1639 int 'int' Loc=<src/main.c:12:10>
1640 identifier 'i' [LeadingSpace] Loc=<src/main.c:12:14>
1641 equal '=' [LeadingSpace] Loc=<src/main.c:12:16>
1642 numeric_constant '0' [LeadingSpace] Loc=<src/main.c:12:18>
1643 semi ';' Loc=<src/main.c:12:19>
1644 identifier 'i' [LeadingSpace] Loc=<src/main.c:12:21>
1645 less '<' [LeadingSpace] Loc=<src/main.c:12:23>
1646 identifier 'count' [LeadingSpace] Loc=<src/main.c:12:25>
1647 semi ';' Loc=<src/main.c:12:30>
1648 plusplus '++' [LeadingSpace] Loc=<src/main.c:12:32>

```

图 35 Clang token dump 及源码位置

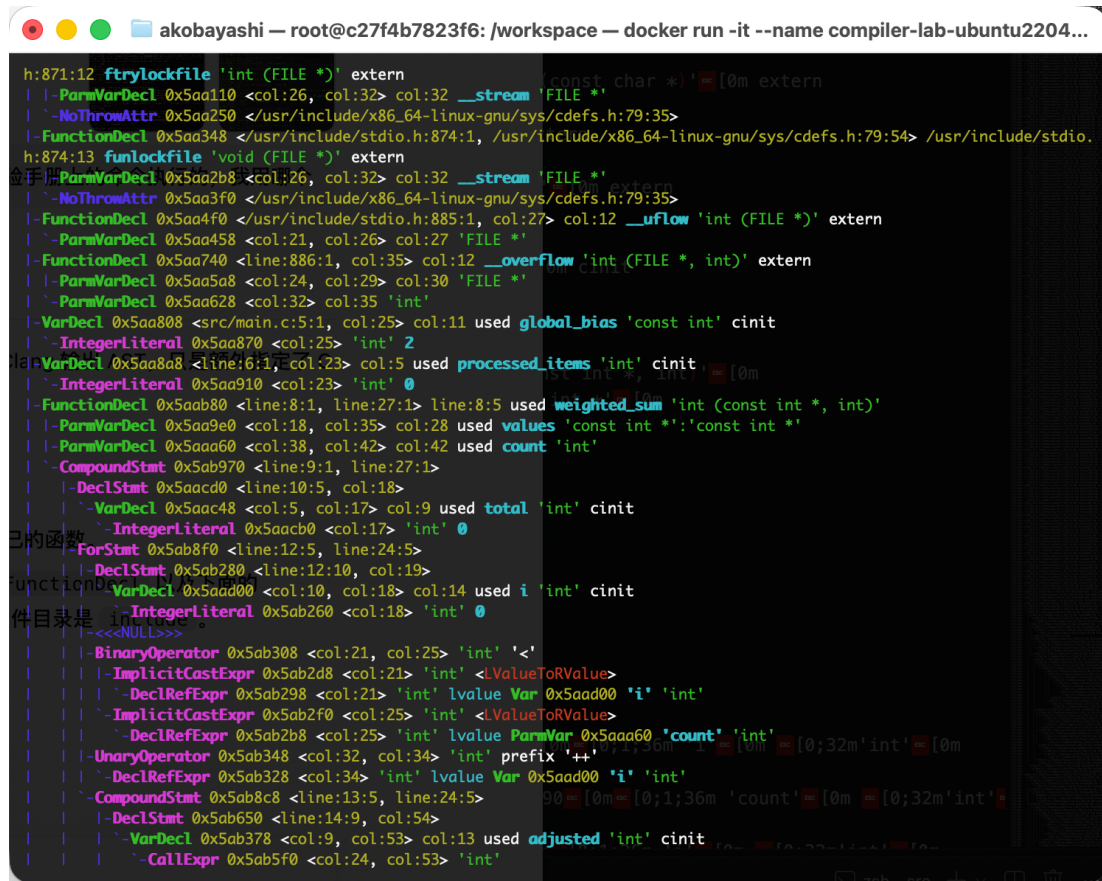


图 36 Clang AST 中的函数、循环与条件节点

4.3 控制流图与 LLVM IR

为直接观察控制流图, GCC 在编译时写出 tree/CFG 图文件, 再由 Graphviz 将 .dot 文件绘制为图 37。其中循环回边对应 for 的重复执行, 条件分支对应奇偶判断以及 main 中的输入合法性检查。

```

1 mkdir -p build/memberB/q1/gcc-graphs
2 gcc -O0 -Ipipeline/memberB/q1/include -fdump-tree-all-graph \
3   -dumpdir build/memberB/q1/gcc-graphs/ -dumpbase main-gcc \
4   -c pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-gcc.o
5 dot -Tpng build/memberB/q1/gcc-graphs/main-gcc.015t.cfg.dot \
6   -o build/memberB/q1/gcc-graphs/main-cfg.png

```

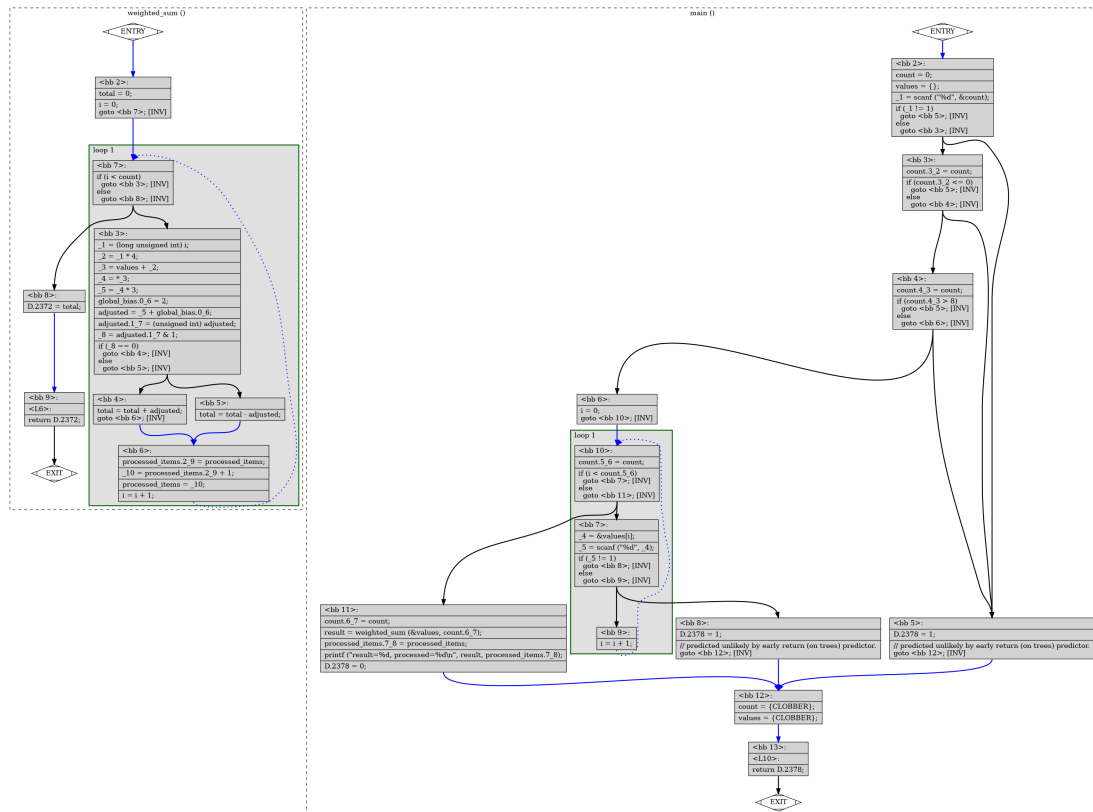


图 37 GCC 生成并由 Graphviz 绘制的控制流图

LLVM IR 由 Clang 前端从同一份 C 源码生成。`-O0` 保留较多局部存储、`load/store` 与显式控制流，便于将 IR 与源码逐句对应；图 38 展示了函数定义、数组操作、比较和分支。

```
1 clang -std=c17 -Ipipeline/memberB/q1/include -O0 -S -emit-llvm \
2   pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-O0.ll
```

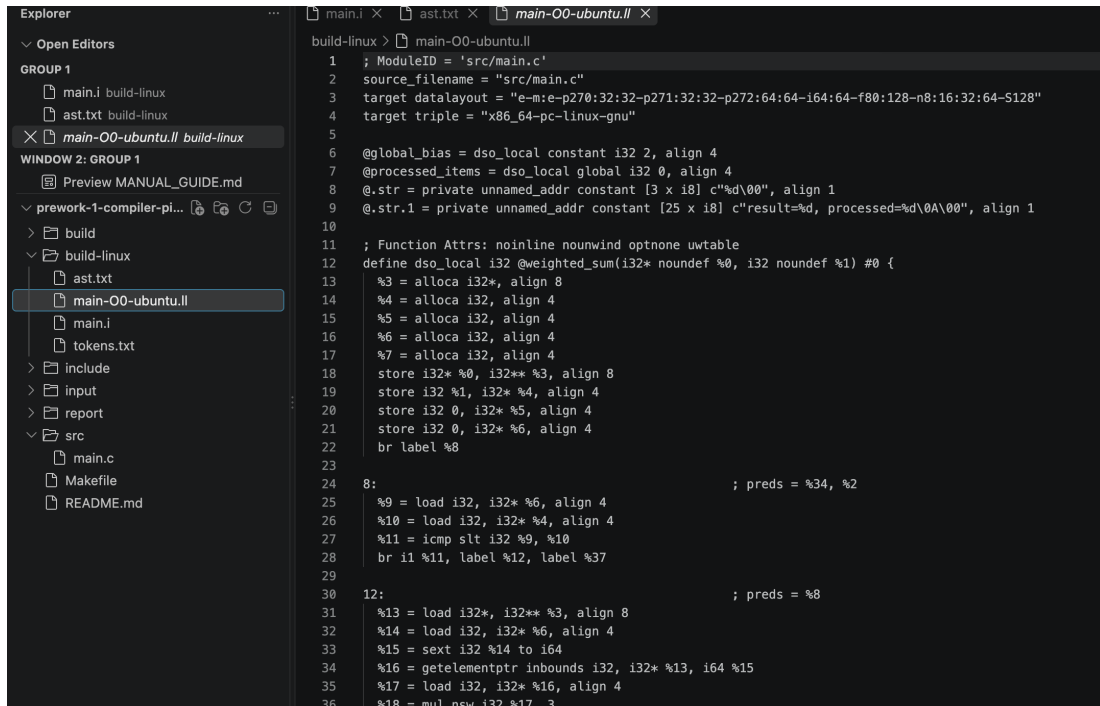


图 38 未优化 LLVM IR：函数、基本块与内存操作

4.4 优化前后比较与目标代码生成

以相同源程序分别使用 `-O0` 和 `-O2` 生成 IR，构成参数对照。图 39 中，优化版本的局部计算更倾向于使用 SSA 临时值，减少部分冗余栈读写，并带有 `nsw` 等运算属性；函数属性、循环组织也有变化。优化并不改变本程序的可观察输入输出，但 IR 不再是源代码逐行翻译的形式。

```

1 clang -std=c17 -Ipipeline/memberB/q1/include -O2 -S -emit-llvm \
2   pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-02.ll

```

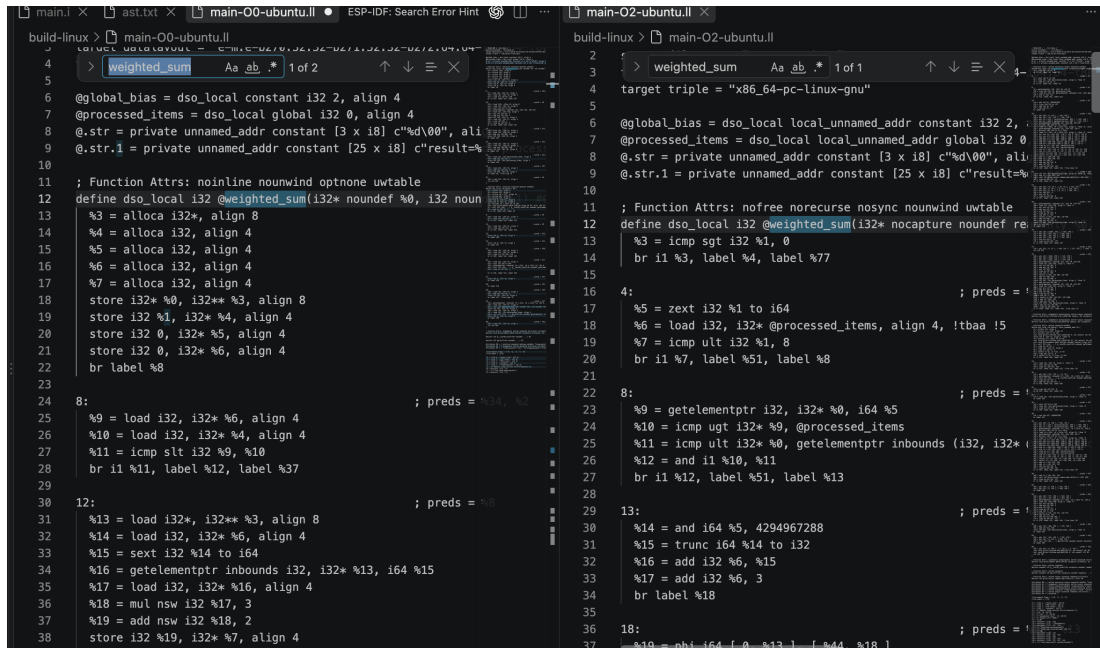


图 39 同一函数在 -O0 与 -O2 下的 LLVM IR 对照

目标代码阶段使用 LLVM 后端将 IR 映射为 AArch64 汇编。该汇编出现了 32 位/64 位通用寄存器、条件分支和数组寻址指令；汇编器再把文本指令编码为 AArch64 机器码。目标文件为可重定位 ELF，仍包含需要链接器处理的符号与重定位项。

```

1 clang --target=aarch64-linux-gnu -std=c17 \
2   -Ipipeline/memberB/q1/include -O2 -S -emit-llvm \
3   pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-02-aarch64.
  ll
4 llc -O2 build/memberB/q1/main-02-aarch64.ll \
5   -o build/memberB/q1/main-02-aarch64.s
6 aarch64-linux-gnu-gcc -c build/memberB/q1/main-02-aarch64.s \
7   -o build/memberB/q1/main-02-aarch64.o
8 aarch64-linux-gnu-readelf -h build/memberB/q1/main-02-aarch64.o
9 aarch64-linux-gnu-readelf -Ws -rW build/memberB/q1/main-02-aarch64.o

```

图 40 是 AArch64 汇编片段；图 41 的 ELF 头中 Type: REL 与 Machine: AArch64 说明产物是尚未链接的可重定位目标文件。图 42 中，自定义函数和全局变量进入符号表，printf/scanf 等外部函数仍是未定义符号；R_AARCH64_* 重定位项记录了链接时需要修补的位置。


```

root@c27f4b7823f6:/workspace/build-linux# aarch64-linux-gnu-readelf -Ws main-02-aarch64.o
[aarch64-linux-gnu-readelf -rW main-02-aarch64.o]

Symbol table '.symtab' contains 20 entries:
  Num:  Value              Size Type    Bind   Vis      Ndx Name
  ---:  ---              ---: ---: ---:   ---: ---:
  0: 0000000000000000      0 NOTYPE  LOCAL  DEFAULT UND
  1: 0000000000000000      0 FILE    LOCAL  DEFAULT ABS main.c
  2: 0000000000000000      0 SECTION LOCAL  DEFAULT 1 .text
  3: 0000000000000000      0 SECTION LOCAL  DEFAULT 3 .data
  4: 0000000000000000      0 SECTION LOCAL  DEFAULT 4 .bss
  5: 0000000000000000      0 NOTYPE  LOCAL  DEFAULT 1 $x
  6: 0000000000000000      0 SECTION LOCAL  DEFAULT 5 .rodata
  7: 0000000000000000      0 NOTYPE  LOCAL  DEFAULT 5 $d
  8: 0000000000000000      0 NOTYPE  LOCAL  DEFAULT 4 $d
  9: 0000000000000000      0 SECTION LOCAL  DEFAULT 6 .rodata.str1.1
 10: 0000000000000000      0 SECTION LOCAL  DEFAULT 8 .note.GNU-stack
 11: 0000000000000014      0 NOTYPE  LOCAL  DEFAULT 9 $d
 12: 0000000000000000      0 SECTION LOCAL  DEFAULT 9 .eh_frame
 13: 0000000000000000      0 SECTION LOCAL  DEFAULT 7 .comment
 14: 0000000000000000    296 FUNC    GLOBAL  DEFAULT 1 weighted_sum
 15: 0000000000000000      4 OBJECT  GLOBAL  DEFAULT 4 processed_items
 16: 0000000000000128    432 FUNC    GLOBAL  DEFAULT 1 main
 17: 0000000000000000      0 NOTYPE  GLOBAL  DEFAULT UND __isoc99_scanf
 18: 0000000000000000      0 NOTYPE  GLOBAL  DEFAULT UND printf
 19: 0000000000000000      4 OBJECT  GLOBAL  DEFAULT 5 global_bias

Relocation section '.rela.text' at offset 0x5e0 contains 22 entries:
  Offset             Info             Type             Symbol's Value Symbol's Name + Addend
  0000000000000008  0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 0
  0000000000000014  0000000f0000011d R_AARCH64_LDST32_ABS_L012_NC 0000000000000000 processed_items + 0
  000000000000001c  0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 0
  0000000000000024  0000000f00000115 R_AARCH64_ADD_ABS_L012_NC 0000000000000000 processed_items + 0
  0000000000000030  0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 4
  0000000000000034  0000000f00000115 R_AARCH64_ADD_ABS_L012_NC 0000000000000000 processed_items + 4
  0000000000000060  0000000f0000011d R_AARCH64_LDST32_ABS_L012_NC 0000000000000000 processed_items + 0
  0000000000000114  0000000f0000011d R_AARCH64_LDST32_ABS_L012_NC 0000000000000000 processed_items + 0
  0000000000000140  0000000900000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 .rodata.str1.1 + 0
  0000000000000144  0000000900000115 R_AARCH64_ADD_ABS_L012_NC 0000000000000000 .rodata.str1.1 + 0
  0000000000000154  000000110000011b R_AARCH64_CALL26 0000000000000000 __isoc99_scanf + 0
  000000000000017c  0000000900000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 .rodata.str1.1 + 0
  0000000000000188  0000000900000115 R_AARCH64_ADD_ABS_L012_NC 0000000000000000 .rodata.str1.1 + 0
  0000000000000194  000000110000011b R_AARCH64_CALL26 0000000000000000 __isoc99_scanf + 0
  00000000000001c0  0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 0
  00000000000001c8  0000000f0000011d R_AARCH64_LDST32_ABS_L012_NC 0000000000000000 processed_items + 0
  00000000000001e4  0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 0
  00000000000001ec  0000000f0000011d R_AARCH64_LDST32_ABS_L012_NC 0000000000000000 processed_items + 0
  00000000000002b0  0000000f0000011d R_AARCH64_LDST32_ABS_L012_NC 0000000000000000 processed_items + 0
  00000000000002b4  0000000900000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 .rodata.str1.1 + 3
  00000000000002b8  0000000900000115 R_AARCH64_ADD_ABS_L012_NC 0000000000000000 .rodata.str1.1 + 3
  00000000000002bc  000000120000011b R_AARCH64_CALL26 0000000000000000 printf + 0

Relocation section '.rela.eh_frame' at offset 0x7f0 contains 2 entries:
  Offset             Info             Type             Symbol's Value Symbol's Name + Addend
  000000000000001c  0000000200000105 R_AARCH64_PREL32 0000000000000000 .text + 0
  0000000000000030  0000000200000105 R_AARCH64_PREL32 0000000000000000 .text + 128
root@c27f4b7823f6:/workspace/build-linux#

```

图 42 目标文件的符号表与重定位表

4.5 链接、反汇编与运行验证

将目标文件与 C 运行时库链接后，外部函数调用可解析为可执行文件中的动态链接入口。图 43 对照了链接前后的反汇编：目标文件中保留针对 `printf` 的调用重定位，链接后的程序则显示具体地址和 `printf@plt` 调用入口。AArch64 可执行文件不能直接作为 macOS arm64 原生程序运行；本实验在 Ubuntu 容器中通过 QEMU 用户态执行，并用同一组输入检查结果。

```
clang -std=c17 -Ipipeline/memberB/q1/include -O2 \
```

```

2 pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-native
3 build/memberB/q1/main-native < pipeline/memberB/q1/input/sample.txt
4 aarch64-linux-gnu-gcc -no-pie build/memberB/q1/main-02-aarch64.o \
5 -o build/memberB/q1/main-aarch64
6 qemu-aarch64 -L /usr/aarch64-linux-gnu \
7 build/memberB/q1/main-aarch64 < pipeline/memberB/q1/input/sample.txt

```

样例数据为 1, 2, 3, 4, 5, 调整值分别为 5, 8, 11, 14, 17, 按奇偶规则计算得到 -11, 处理数为 5。报告截图中的目标文件/链接证据来自 AArch64 目标; 以上原生编译命令用于验证同一 C 源程序的运行语义。

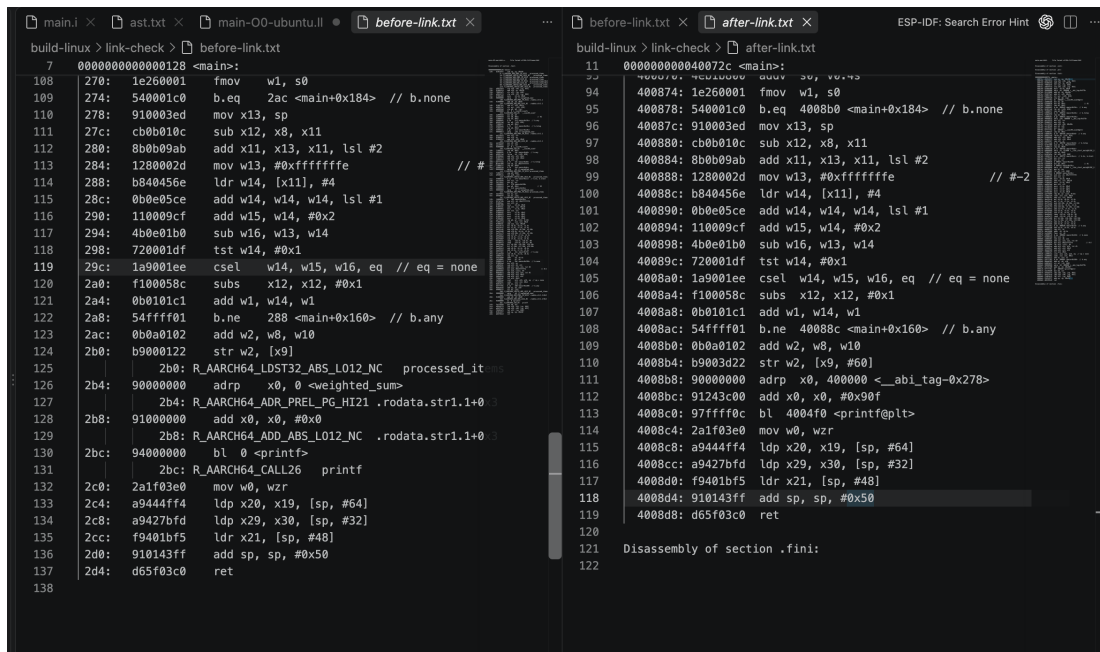


图 43 链接前目标文件与链接后程序的反汇编对照

4.6 阶段观察小结

本实验使用不同产物观察同一程序：预处理文件体现宏展开，token 与 AST 体现前端识别出的词法单元和语法结构，CFG/LLVM IR 体现循环、条件和数据流，优化前后 IR 体现中间表示变换，AArch64 汇编与 ELF 目标文件体现后端及汇编器的结果，符号/重定位和链接后反汇编则揭示外部库调用如何延迟到链接阶段处理。-00 与 -02 是本次实际比较的编译参数扩展。预处理输出与运行输出已按命令和源程序记录；当前图片集未包含这两项的独立截图，因此不将其描述为截图证据。

5 小题 2: SysY 等价 LLVM IR 与 RISC-V 程序实验

小题 2 由两名成员分别完成 LLVM IR 与目标汇编两个任务。两部分共享同一份 SysY 示例程序和同一组测试数据,使两种实现围绕统一语义基准进行比较。SysY 程序由两人共同确定;成员 B 负责 LLVM IR,成员 A 负责 RISC-V 汇编。

5.1 共同 SysY 示例程序设计

SysY 是由 C 语言子集扩展而来的教学语言。本实验设计的程序读入 n 个整数,计算其中所有偶数之和,并统计正数个数。程序包含数组、赋值、算术与取模运算、关系判断、if 条件分支、while 循环、函数、数组参数和函数调用等语言特性;输入输出通过 SysY 运行时库中的 `getint`、`putint` 和 `putch` 完成。为保证数组访问范围有效,测试时取 $0 \leq n \leq 10$ 。

```
1 //求偶数和
2 int sum_even(int a[],int n)
3 {
4     int i;
5     int sum;
6     i=0;
7     sum=0;
8     while(i<n)
9     {
10         if(a[i]%2==0)
11             sum=sum+a[i];
12         i=i+1;
13     }
14     return sum;
15 }
16
17 //统计正数个数
18 int count_pos(int a[],int n)
19 {
20     int i;
21     int cnt;
22     i=0;
23     cnt=0;
24     while(i<n)
25     {
26         if(a[i]>0)
27             cnt=cnt+1;
```

```
28     i=i+1;
29 }
30 return cnt;
31 }
32
33 int main()
34 {
35     int a[10];
36     int n;
37     int i;
38     int s;
39     int c;
40
41     //输入
42     n=getint();
43     i=0;
44     while(i<n)
45     {
46         a[i]=getint();
47         i=i+1;
48     }
49
50     //计算
51     s=sum_even(a,n);
52     c=count_pos(a,n);
53
54     //输出
55     putint(s);
56     putch(10);
57     putint(c);
58     putch(10);
59
60     return 0;
61 }
```

代码 3 SysY 示例程序 main.sy

为便于两种实现统一验证，固定使用三组测试数据。第 1 组同时包含正数、负数、奇数和偶数；第 2 组验证负偶数参与求和；第 3 组全部为正奇数，用于验证偶数和为 0 的情况。两名成员的 LLVM IR 与 RISC-V 实现均应使用这三组输入并得到相同输出。

表 2 小题 2 统一测试数据

组别	输入 (第一项为 n)	偶数和	正数个数
1	5; 1 2 -3 4 6	12	4
2	4; -2 3 8 -5	6	2
3	5; 1 3 5 7 9	0	5

5.2 LLVM IR 等价实现 (成员 B 负责)

本节以代码 3 中的小题 2 共享 SysY 程序为语义基准, 手工编写 LLVM IR, 而不是由 C/SysY 编译器自动生成。IR 保持三个函数 `sum_even`、`count_pos` 和 `main`; 它使用 SSA 临时值和基本块表示循环与分支, 并调用课程运行时库完成输入输出。文件位于 `llvm/memberB/main.ll`。

5.2.1 源程序结构到 LLVM IR 的映射

表 3 SysY 结构与手写 LLVM IR 的对应关系

SysY 结构	LLVM IR 指令/结构	含义
<code>int a[10]</code>	<code>alloca [10 x i32]</code>	在 <code>main</code> 栈帧分配 10 个 32 位整数
<code>a[i]</code>	<code>getelementptr, load/store</code>	计算元素地址并读写内存
<code>while (i<n)</code>	<code>br, icmp slt</code>	比较循环条件并在基本块间跳转
<code>a[i] % 2 == 0</code>	<code>srem, icmp eq</code>	有符号取余后判断是否为偶数
<code>if / else</code>	条件 <code>br</code> 与 <code>phi</code>	分支选路并合并更新后的累加值
函数参数/返回值	<code>i32*, i32, ret</code>	数组首地址和长度作为参数, 结果作为整数返回
输入/输出	<code>call</code>	调用 SysY 运行时库的 <code>getint</code> 、 <code>putint</code> 、 <code>putch</code> 接口

在 `sum_even` 中, 循环头的 `phi` 节点选择初值 0 或上一轮的下标/和; 偶数分支计算新的和, 不满足条件的分支保留原和, 再由另一个 `phi` 合并。`count_pos` 采用相同结构统计正数。`main` 在栈上分配数组, 通过 `getint` 逐项读入, 再把首元素地址和 n 传给两个计算函数。上述控制流和数据流均为手工编写的 LLVM IR。

5.2.2 完整 LLVM IR 程序

```

1 ; Hand-written LLVM IR equivalent to report/sections/q2_common_sysy.
   tex.
2 ; The SysY runtime provides getint, putint and putch.
```

```
3
4 target triple = "x86_64-pc-linux-gnu"
5
6 declare i32 @getint()
7 declare void @putint(i32)
8 declare void @putch(i32)
9
10 define i32 @sum_even(i32* %a, i32 %n) {
11 entry:
12   br label %loop.cond
13
14 loop.cond:
15   %i = phi i32 [ 0, %entry ], [ %i.next, %if.end ]
16   %sum = phi i32 [ 0, %entry ], [ %sum.next, %if.end ]
17   %more = icmp slt i32 %i, %n
18   br i1 %more, label %loop.body, label %loop.exit
19
20 loop.body:
21   %element.ptr = getelementptr inbounds i32, i32* %a, i32 %i
22   %value = load i32, i32* %element.ptr, align 4
23   %remainder = srem i32 %value, 2
24   %is.even = icmp eq i32 %remainder, 0
25   br i1 %is.even, label %if.then, label %if.else
26
27 if.then:
28   %sum.add = add nsw i32 %sum, %value
29   br label %if.end
30
31 if.else:
32   br label %if.end
33
34 if.end:
35   %sum.next = phi i32 [ %sum.add, %if.then ], [ %sum, %if.else ]
36   %i.next = add nsw i32 %i, 1
37   br label %loop.cond
38
39 loop.exit:
40   ret i32 %sum
41 }
42
43 define i32 @count_pos(i32* %a, i32 %n) {
```

```
44 entry:
45     br label %loop.cond
46
47 loop.cond:
48     %i = phi i32 [ 0, %entry ], [ %i.next, %if.end ]
49     %count = phi i32 [ 0, %entry ], [ %count.next, %if.end ]
50     %more = icmp slt i32 %i, %n
51     br i1 %more, label %loop.body, label %loop.exit
52
53 loop.body:
54     %element.ptr = getelementptr inbounds i32, i32* %a, i32 %i
55     %value = load i32, i32* %element.ptr, align 4
56     %is.positive = icmp sgt i32 %value, 0
57     br i1 %is.positive, label %if.then, label %if.else
58
59 if.then:
60     %count.inc = add nsw i32 %count, 1
61     br label %if.end
62
63 if.else:
64     br label %if.end
65
66 if.end:
67     %count.next = phi i32 [ %count.inc, %if.then ], [ %count, %if.else ]
68     %i.next = add nsw i32 %i, 1
69     br label %loop.cond
70
71 loop.exit:
72     ret i32 %count
73 }
74
75 define i32 @main() {
76 entry:
77     %a = alloca [10 x i32], align 16
78     %n = call i32 @getint()
79     br label %input.cond
80
81 input.cond:
82     %i = phi i32 [ 0, %entry ], [ %i.next, %input.body ]
83     %more = icmp slt i32 %i, %n
84     br i1 %more, label %input.body, label %compute
```

```

85
86 input.body:
87   %element.ptr = getelementptr inbounds [10 x i32], [10 x i32]* %a,
      i32 0, i32 %i
88   %value = call i32 @getint()
89   store i32 %value, i32* %element.ptr, align 4
90   %i.next = add nsw i32 %i, 1
91   br label %input.cond
92
93 compute:
94   %first = getelementptr inbounds [10 x i32], [10 x i32]* %a, i32 0,
      i32 0
95   %sum = call i32 @sum_even(i32* %first, i32 %n)
96   %positive = call i32 @count_pos(i32* %first, i32 %n)
97   call void @putint(i32 %sum)
98   call void @putch(i32 10)
99   call void @putint(i32 %positive)
100  call void @putch(i32 10)
101  ret i32 0
102 }

```

代码 4 手写 LLVM IR：与共享 SysY 程序等价的 main.ll

5.2.3 汇编、目标文件与运行时库链接

实验固定在 Ubuntu 22.04、LLVM/Clang 14 容器中完成，IR 的 target triple 为 x86_64-pc-linux-gnu。课程运行时实现使用仓库现有的 riscv/lib/sylib.c；这里只复用其标准 C 源码编译成本机共享库/目标文件，不修改或覆盖成员 A 的 RISC-V 目标库。仓库内的 llvm/memberB/Makefile 将验证产物隔离在本目录下的 build/：

```

1 cd llvm/memberB
2 make all
3 llvm-readelf -h build/main.o
4 llvm-readelf -rW build/main.o
5 nm build/main-native | grep -E "getint|putint|putch|sum_even|count_pos
  | main$"
6 make test

```

llvm-as 将文本 IR 汇编为 bitcode，用以检查 IR 语法；Clang 把同一 IR 降低为可重定位本机目标文件，并与运行时库对象链接成原生可执行文件。另将 sylib.c 编译为共

享库，交由 `lli -load=...` 载入，从而验证 IR 也可由 LLVM JIT 执行。目标文件中的运行时符号由 `nm` 检查。

5.2.4 三组统一测试结果

分别以 `make test-native` 和 `make test-lli` 执行统一测试集。每组数据分别经由 Clang 生成的本机可执行文件和 LLVM `lli` 执行；SysY 运行时在标准错误输出额外打印 TOTAL 计时行，表中只列出程序标准输出的两个计算结果。

表 4 LLVM IR 原生执行与 lli 执行结果

组别	输入	预期输出	native / lli 实际输出
1	5; 1 2 -3 4 6	12; 4	12; 4 (两种方式均通过)
2	4; -2 3 8 -5	6; 2	6; 2 (两种方式均通过)
3	5; 1 3 5 7 9	0; 5	0; 5 (两种方式均通过)

三组输入的原生执行与 `lli` 执行输出一致，且与共享 SysY 源程序的预期值相符。第 2 组验证负偶数保留其代数符号参与求和；第 3 组验证没有偶数时累加结果为 0。本节保留了可重现的 IR、运行时链接 Makefile 和输入文件；目前提供的 `image/` 目录不含本节执行画面的独立截图，因此结果以可重跑命令和表格记录，不将文字结果冒充为截图。

5.2.5 LLVM IR 部分小结

手写 IR 中, `br` 将 `while` 和 `if/else` 转换为显式基本块控制流; `getelementptr` 根据数组首地址和索引计算元素地址, `load/store` 读写数组元素; `phi` 在控制流汇合点选择不同前驱基本块产生的 SSA 值; `call/ret` 表示运行时函数、计算函数的调用和返回。与 RISC-V 汇编相比, LLVM IR 保留类型、函数和控制流等较高层次结构, 暂不指定通用寄存器及具体指令编码; 后端再根据 `target triple` 将 IR 映射到目标架构。

5.3 RISC-V 等价实现 (成员 A 负责)

5.3.1 SysY 到 RISC-V 的实现对应

RISC-V 版本保持与 `main.sy` 相同的函数划分和控制逻辑。按照 RV64 调用约定, 整数和地址参数通过 `a0`、`a1` 等参数寄存器传递, 整数返回值放在 `a0` 中; `main` 中需要跨函数调用保存的 `n`、循环变量和数组基址分别放入 `s0`、`s1`、`s2`, 并在栈上保存 `ra` 和这些被调用者保存寄存器。栈帧大小取 80 B, 为 16 的整数倍。

表 5 SysY 结构与 RISC-V 实现的对应关系

SysY 结构	RISC-V 实现	作用
函数参数 a、n	a0、a1	按照调用约定传递数组首地址和整数参数
return	a0	函数返回值寄存器
a[i]	slli + add + lw/sw	将下标左移 2 位得到 $i \times 4$ ，再与数组基地址相加
while	条件分支 + 标签 + 回跳	将循环转换为基本块和跳转
a[i]%2==0	remw + bnez	求余后判断是否为 0
a[i]>0	blez	非正数时跳过计数
getint/putint/call		调用 SysY 运行时库函数
putch		
main 栈帧	addi sp,sp,-80 / +80	保存 ra 和 s0-s2，并保持 16 B 对齐

汇编中使用了 li、mv、j、call、ret、bnez、blez 等常见伪指令，汇编器会将其展开为相应的基础指令组合。手写汇编的重点不是逐字翻译源代码，而是保持函数接口、控制流、数据访问和最终运行结果与 SysY 程序等价。

5.3.2 RISC-V 汇编程序

```

1 .option nopic
2 .attribute stack_align,16
3 .text
4
5 .globl sum_even
6 .type sum_even,@function
7 sum_even:
8 #初始化
9     li t0,0
10    li t1,0
11 .Lse_loop:
12    bge t0,a1,.Lse_end
13    slli t2,t0,2
14    add t3,a0,t2
15    lw t4,0(t3)
16 #判断偶数
17    li t5,2
18    remw t6,t4,t5
19    bnez t6,.Lse_next
20    addw t1,t1,t4

```

```
21 .Lse_next:
22     addi t0,t0,1
23     j .Lse_loop
24 .Lse_end:
25     mv a0,t1
26     ret
27 .size sum_even,.-sum_even
28
29 .globl count_pos
30 .type count_pos,@function
31 count_pos:
32 #初始化
33     li t0,0
34     li t1,0
35 .Lcp_loop:
36     bge t0,a1,.Lcp_end
37     slli t2,t0,2
38     add t3,a0,t2
39     lw t4,0(t3)
40 #统计正数
41     blez t4,.Lcp_next
42     addiw t1,t1,1
43 .Lcp_next:
44     addi t0,t0,1
45     j .Lcp_loop
46 .Lcp_end:
47     mv a0,t1
48     ret
49 .size count_pos,.-count_pos
50
51 .globl main
52 .type main,@function
53 main:
54 #开辟栈空间
55     addi sp,sp,-80
56     sd ra,72(sp)
57     sd s0,64(sp)
58     sd s1,56(sp)
59     sd s2,48(sp)
60
61     mv s2,sp
```

```
62
63 #输入n
64     call getint
65     mv s0,a0
66     li s1,0
67
68 #输入数组
69 .Linput:
70     bge s1,s0,.Linput_end
71     call getint
72     slli t0,s1,2
73     add t0,s2,t0
74     sw a0,0(t0)
75     addi s1,s1,1
76     j .Linput
77
78 .Linput_end:
79 #求偶数和
80     mv a0,s2
81     mv a1,s0
82     call sum_even
83     call putint
84     li a0,10
85     call putch
86
87 #统计正数
88     mv a0,s2
89     mv a1,s0
90     call count_pos
91     call putint
92     li a0,10
93     call putch
94
95     li a0,0
96     ld s2,48(sp)
97     ld s1,56(sp)
98     ld s0,64(sp)
99     ld ra,72(sp)
100    addi sp,sp,80
101    ret
102 .size main,.-main
```

代码 5 与 main.sy 等价的 RV64 汇编程序 main.s

5.3.3 汇编与目标文件生成

首先使用 RISC-V 交叉编译器的汇编阶段把 main.s 转换为可重定位目标文件 main.o:

```
1 riscv64-unknown-elf-gcc main.s -c -o main.o -w
2 file main.o
```

file 输出中包含 ELF 64-bit、relocatable 和 UCB RISC-V, 说明文本汇编已被编码为 64 位 RISC-V 目标文件, 但此时仍处于可重定位状态, 尚不能独立运行。

5.3.4 链接 SysY 运行时库

程序中的 getint、putint 和 putch 不是在 main.s 中实现的, 因此需要链接课程提供的 SysY RISC-V 运行时库 libsysy_riscv.a。实验采用静态链接和 medany 内存模型, 并按课程环境把代码基址设置为 0x90000000:

```
1 riscv64-unknown-elf-gcc main.o -o main \
2   -L./lib -lsysy_riscv \
3   -static -mcmodel=medany \
4   -Wl,--no-relax,-Ttext=0x90000000
```

首次尝试链接时, 终端优先使用了较旧的 SiFive GCC-Metal 10.2.0 工具链, 其链接器无法识别官方运行库目标文件中的 zmmul ISA 属性。检查 type -a 后发现课程工具链已经安装在 /home/lenovo/riscv-elf-toolchains/bin, 将该目录置于 PATH 前端并重新生成 main.o 后, 官方 libsysy_riscv.a 可以正常链接。该现象说明预编译静态库中的目标架构属性需要与所使用的汇编器、链接器版本兼容。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ cd /mnt/d/compilerLab/riscv
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ rm -f main.o main
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ riscv64-unknown-elf-gcc main.s -c -o main.o -w
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ file main.o
main.o: ELF 64-bit LSB relocatable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ riscv64-unknown-elf-gcc main.o -o main \
-L./lib -lsysy_riscv \
-static -mcmodel=medany \
-Wl,--no-relax,-Ttext=0x90000000
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$

```

图 44 RV64 目标文件检查与 SysY 运行时库链接

5.3.5 运行结果验证

链接成功后, file 显示 main 为 ELF 64-bit LSB executable, UCB RISC-V, 并且采用静态链接。使用 qemu-riscv64 执行程序, 对正常数据、包含负数的数据以及全奇数数据分别进行测试。运行时库额外打印的 TOTAL 计时行由 SysY 运行时库产生, 不属于本程序的两个计算结果。

```
1 qemu-riscv64 ./main
```

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ file main
ls -lh main
main: ELF 64-bit LSB executable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), statically linked, with debug_info, not stripped
-rwxrwxrwx 1 lenovo lenovo 164K Sep 25 02:33 main
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ qemu-riscv64 ./main
5
1
2
-3
4
6
12
4
TOTAL: 0H-0M-0S-0us
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$

```

图 45 可执行文件类型与第 1 组运行结果

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ qemu-riscv64 ./main
4
-2
3
8
-5
6
2
TOTAL: 0H-0M-0S-0us
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$
```

图 46 第 2 组运行结果（含负数）

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ qemu-riscv64 ./main
5
1
3
5
7
9
0
5
TOTAL: 0H-0M-0S-0us
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$
```

图 47 第 3 组运行结果（全奇数）

表 6 RISC-V 程序多组测试结果

组别	输入	预期输出	实际输出	结果
1	5; 1 2 -3 4 6	12; 4	12; 4	正确
2	4; -2 3 8 -5	6; 2	6; 2	正确
3	5; 1 3 5 7 9	0; 5	0; 5	正确

三组测试的实际输出均与对 SysY 源程序的直接计算结果一致。其中第 2 组覆盖了负数参与偶数求和的情况，第 3 组使偶数和为 0，可进一步验证条件分支和循环控制没有只针对单一输入工作。

5.3.6 运行时库链接验证

为了进一步确认最终可执行文件确实包含 SysY 运行时库，使用 nm 查看链接后的符号表：

```
1 riscv64-unknown-elf-nm main \
2 | grep -E "getint|putint|putch|sum_even|count_pos| main$"
```

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ riscv64-unknown-elf-nm ma
in | grep -E "getint|putint|putch|sum_even|count_pos| main$"
000000009000038a0 T _putchar_r
0000000090000134 T count_pos
00000000900001d2 T getint
0000000090000156 T main
0000000090000360 T putch
000000009000038aa T putchar
0000000090000332 T putint
000000009000010a T sum_even
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$

```

图 48 最终可执行文件中的自定义函数与 SysY 运行时库符号

符号表中同时存在手写汇编定义的 `sum_even`、`count_pos`、`main`，以及运行时库提供的 `getint`、`putint`、`putch`。由此可以确认最终程序不仅在运行结果上正确，而且确实完成了题目要求的“RISC-V 汇编程序链接 SysY 运行库”。

5.3.7 RISC-V 部分小结

本实验从一份包含数组、赋值、运算、条件分支、循环和函数等语言特性的 SysY 程序出发，手工完成了等价 RV64 汇编实现。数组元素访问被转换为地址计算和 `lw/sw`，`while` 与 `if` 被转换为条件分支和标签跳转，函数参数和返回值按照 RISC-V 调用约定通过 `a0`、`a1` 等寄存器传递。`main.s` 经汇编得到可重定位目标文件，再与官方 `libsysy_riscv.a` 静态链接得到 RISC-V 可执行文件。三组输入在 QEMU 下均得到预期结果，`nm` 也验证了 SysY 运行时库函数已经进入最终可执行文件，说明本部分完成了从 SysY 语义到 RISC-V 目标程序的等价实现与运行验证。

6 结论

本实验围绕语言处理系统的完整工作过程和 SysY 程序的等价实现进行了分阶段研究。成员 A 的小题 1 实验表明，预处理阶段完成头文件展开、宏替换与条件编译；编译器内部可以通过 `token`、`tree`、`CFG`、`RTL` 和汇编代码观察词法、语法、语义、中间表示、优化与代码生成过程；汇编器将文本汇编转换为可重定位目标文件，链接器再解析符号和库依赖形成最终可执行文件。对 `-DDEBUG`、`-O2`、`-g`、`-fno-omit-frame-pointer`、`-static` 和自动并行化选项的对照还说明，命令行参数会直接影响不同阶段的输出形式和最终程序组织。

小题 2 采用同一份 SysY 程序作为语义基准。已完成的 RISC-V 部分将数组访问、循环、条件分支、函数调用和运行时输入输出映射为 RV64 指令与调用约定，并成功链接 SysY 运行时库，在 QEMU 下通过三组输入得到预期结果。该过程说明，与源程序等价不仅要求计算结果一致，还需要正确处理参数传递、数组地址计算、控制流、栈帧

以及外部运行时函数等低层问题。

成员 B 的小题 1 进一步通过 Clang token/AST、GCC 控制流图、LLVM IR 优化前后对照以及 AArch64 ELF 符号和重定位信息观察了编译阶段间的转换；小题 2 的手写 IR 则用基本块、SSA 临时值、phi、getelementptr、load/store、br 和 call 显式表达共享 SysY 程序中的数组、循环、分支和运行时调用。三组统一测试在 IR 原生可执行文件、LLVM lli 和成员 A 的 RISC-V/QEMU 路径中均得到预期结果。LLVM IR 的抽象层次高于 RISC-V 汇编：它保留类型与显式控制流，但不指定具体通用寄存器和机器指令编码；RISC-V 实现则需要直接遵循目标架构的寄存器、指令和调用约定。两种实现展示了同一源程序语义向中间表示和机器相关目标代码的不同映射方式。

参考文献

- [1] 编译原理课程组. 了解编译器、LLVM IR 编程及汇编编程实验指导 [课程资料].
- [2] 编译原理课程组. 编译器开发环境部署 [课程资料].
- [3] SysY 语言工作组. SysY 语言定义（2022 版） [课程资料].
- [4] SysY 语言工作组. SysY 运行时库说明 [课程资料].
- [5] Free Software Foundation. *Using the GNU Compiler Collection (GCC)*. GCC Online Documentation.
- [6] LLVM Project. *LLVM Language Reference Manual*.
- [7] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*.